

A geometric discretization and a simple implementation for variational mesh generation and adaptation*

Weizhang Huang[†] Lennard Kamenski[‡]

November 30, 2015

Abstract

We present a simple direct discretization for functionals used in the variational mesh generation and adaptation. Meshing functionals are discretized on simplicial meshes and the Jacobian matrix of the continuous coordinate transformation is approximated by the Jacobian matrices of affine mappings between elements. The advantage of this direct geometric discretization is that it preserves the basic geometric structure of the continuous functional, which is useful in preventing strong decoupling or loss of integral constraints satisfied by the functional. Moreover, the discretized functional is a function of the coordinates of mesh vertices and its derivatives have a simple analytical form, which allows a simple implementation of variational mesh generation and adaptation on computer. Since the variational mesh adaptation is the base for a number of adaptive moving mesh and mesh smoothing methods, the result in this work can be used to develop simple implementations of those methods. Numerical examples are given.

Key words: variational mesh generation, mesh adaptation, moving mesh

AMS subject classifications: 65N50, 65K10

1. Introduction

The basic idea of the variational approach of mesh generation and adaptation is to generate an adaptive mesh as an image of a given reference mesh under a coordinate transformation determined by a functional (which will hereafter be referred to as a meshing functional). Typically, the meshing functional measures difficulties of the numerical approximation of the physical solution and involves a user-prescribed metric tensor or a monitor function to control the mesh adaptation. The advantage of the variational approach is the relative ease of incorporating mesh requirements such as smoothness, adaptivity, or alignment in the formulation of the functional [2]. The variational approach is commonly used to generate

*Supported in part by the NSF (U.S.A.) under Grant DMS-1115118.

[†]The University of Kansas, Department of Mathematics, Lawrence, KS 66045, U.S.A. (whuang@ku.edu).

[‡]Weierstrass Institute for Applied Analysis and Stochastics, Berlin, Germany (kamenski@wias-berlin.de).

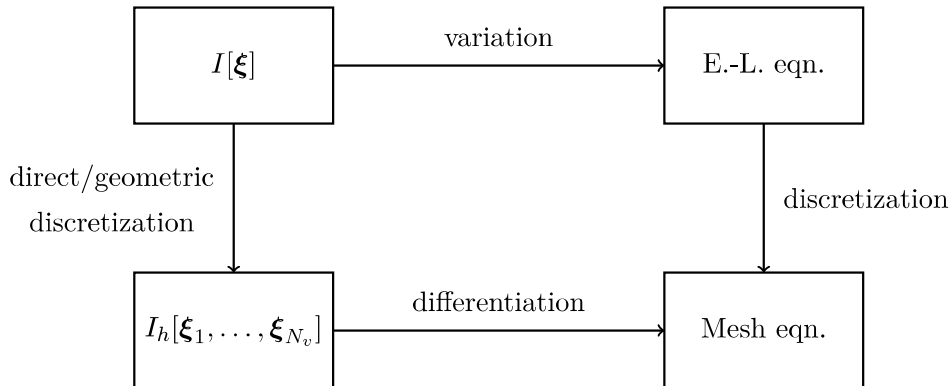


Figure 1: Possible solution strategies for variational mesh generation and adaptation

structured meshes but it can be employed to generate unstructured meshes as well [3]. Moreover, it is the base for a number of adaptive moving mesh methods [11, 12, 14, 20].

A number of variational methods have been developed in the past; e.g., see Thompson et al. [22], Knupp and Steinberg [19], Liseikin [21], Huang and Russell [11] and references therein. Noticeably, Winslow [23] proposed an equipotential method based on variable diffusion. Brackbill and Saltzman [2] developed a method by combining mesh concentration, smoothness, and orthogonality. Dvinsky [5] used the energy of harmonic mappings as his meshing functional. Knupp [16] and Knupp and Robidoux [17] developed functionals based on the idea of conditioning the Jacobian matrix of the coordinate transformation. Huang [9] and Huang and Russell [11] developed functionals based on the so-called equidistribution and alignment conditions.

A common solution strategy for the existing variational methods is to first derive the Euler-Lagrange equation of the underlying meshing functional and then discretize it on either a physical or a computational mesh (cf. Fig. 1). If the discretization is done on a computational mesh, the Euler-Lagrange equation needs to be transformed by changing the roles of dependent and independent variables. Although this strategy works well for the most cases, the corresponding formulation can become complicated and its implementation requires a serious effort, especially in three dimensions; cf. [11, Chapt. 6]. Moreover, the geometric structure of the meshing functional can be lost in the process of spatial discretization of the Euler-Lagrange equation.

The objective of this paper is to study a new discretization and solution strategy. We consider simplicial meshes and approximate the underlying meshing functional directly. Although the direct discretization of the variational problems is not new on itself, its employment in the context of variational mesh generation and adaptation is new. The Jacobian matrix of the coordinate transformation involved in the meshing functional is not discretized directly; instead, it is approximated by the Jacobian matrices of affine mappings between simplicial elements. The advantage of this geometric discretization is that it preserves the basic geometric structure of the continuous functional, which is useful in preventing strong decoupling or loss of integral constraints satisfied by the underlying functional (cf. Castillo [4]). In particular, it preserves the coercivity and convexity for two examples of meshing functionals we consider (see Sect. 3.4). Moreover, as we will

see in Sect. 3, the discretized functional is a function of the coordinates of vertices of the computational mesh and its derivatives have a simple analytical form. This allows a simple (and parallel) implementation of the corresponding variational meshing method.

The outline of the paper is as follows. Section 2 briefly describes the variational approach in mesh generation. Section 3 presents the direct discretization for meshing functionals and gives the analytical formula for the derivatives of the discretized functional with respect to the computational coordinates of mesh vertices. Several numerical examples are presented in Sect. 4, followed by conclusions and further remarks in Sect. 5. For completeness and for the convenience of users who prefer the physical coordinates as unknown variables, the derivatives of the discretized functional with respect to the physical coordinates are given in Appendix A.

2. The variational approach for mesh generation

Let Ω and Ω_c be the physical and computational domains in $\mathbb{R}^d$ ($d \geq 1$), which are assumed to be bounded, simply connected, and polygonal/polyhedral. Generally speaking, Ω_c can be chosen to be the same as Ω but there are benefits to choose it to be convex, including that the to-be-determined coordinate transformation is less likely to be singular, see Dvinsky [5]. We also assume that we are given a symmetric and uniformly positive definite metric tensor $\mathbb{M} = \mathbb{M}(\mathbf{x})$ in Ω , which provides the information about the size and shape of mesh elements. Typically, $\mathbb{M}$ is defined in a mesh adaptation process based on the physical solution, solution error, or other physical considerations.

Denote the coordinates on Ω and Ω_c by $\mathbf{x}$ and $\boldsymbol{\xi}$, the corresponding coordinate transformation by $\mathbf{x} = \mathbf{x}(\boldsymbol{\xi}): \Omega_c \rightarrow \Omega$ and its inverse by $\boldsymbol{\xi} = \boldsymbol{\xi}(\mathbf{x}): \Omega \rightarrow \Omega_c$. Meshing functionals are commonly formulated in terms of the inverse coordinate transformation because the coordinate transformation determined in this way is less likely to be singular [5].

We consider a general meshing functional

$$I[\boldsymbol{\xi}] = \int_{\Omega} G(\mathbb{J}, \det(\mathbb{J}), \mathbb{M}, \mathbf{x}) d\mathbf{x}, \quad (1)$$

where $\mathbb{J} = \frac{\partial \boldsymbol{\xi}}{\partial \mathbf{x}}$ is the Jacobian matrix of $\boldsymbol{\xi} = \boldsymbol{\xi}(\mathbf{x})$ and G is a given smooth function (with respect to all of its arguments). This form is very general and includes many existing meshing functionals, e.g., see Knupp and Steinberg [19], Liseikin [21], and Huang and Russell [11]. To be instructive, we consider two examples in the following. (For a detailed numerical comparison of various functionals see [15].)

Example 2.1 (generalized Winslow's functional). The first example is a generalization of Winslow's variable diffusion functional [23],

$$I[\boldsymbol{\xi}] = \int_{\Omega} \text{tr}(\mathbb{J}\mathbb{M}^{-1}\mathbb{J}^T) d\mathbf{x}, \quad (2)$$

where $\text{tr}(\cdot)$ denotes the trace of a matrix. This functional has been used by many researchers, e.g., see Huang and Russell [10, 12], Li et al. [20], and Beckett et al. [1]. It is coercive and convex and therefore has a unique minimizer [11, Example 6.2.1]. $\square$

Example 2.2 (Huang’s functional). The second functional is

$$I[\boldsymbol{\xi}] = \theta \int_{\Omega} \sqrt{\det(\mathbb{M})} \left(\text{tr}(\mathbb{J} \mathbb{M}^{-1} \mathbb{J}^T) \right)^{\frac{dp}{2}} d\mathbf{x} + (1 - 2\theta) d^{\frac{dp}{2}} \int_{\Omega} \sqrt{\det(\mathbb{M})} \left(\frac{\det(\mathbb{J})}{\sqrt{\det(\mathbb{M})}} \right)^p d\mathbf{x}, \quad (3)$$

where $0 \leq \theta \leq 1$ and $p > 0$ are dimensionless parameters. This functional was proposed by Huang [9] based on the so-called alignment (first term) and equidistribution (second term) conditions. For $0 < \theta \leq \frac{1}{2}$, $dp \geq 2$, and $p \geq 1$, the functional is coercive and polyconvex and has a minimizer [11, Example 6.2.2]. Moreover, for $\theta = \frac{1}{2}$ and $dp = 2$ it reduces to

$$I[\boldsymbol{\xi}] = \theta \int_{\Omega} \sqrt{\det(\mathbb{M})} \text{tr}(\mathbb{J} \mathbb{M}^{-1} \mathbb{J}^T) d\mathbf{x},$$

which is the energy functional for a harmonic mapping from Ω to Ω_c (cf. Dvinsky [5]). $\square$

As mentioned in the introduction, a common strategy to compute the coordinate transformation for a given functional is to discretize and solve its Euler-Lagrange equation. The derivation of the Euler-Lagrange equation for (1) is standard (e.g., see [11, Chapter 6]). It reads as

$$\frac{\delta I}{\delta \boldsymbol{\xi}} \equiv -\nabla \cdot \left(\frac{\partial G}{\partial \mathbb{J}} + \frac{\partial G}{\partial r} \det(\mathbb{J}) \mathbb{J}^{-1} \right) = 0, \quad (4)$$

where $\frac{\delta I}{\delta \boldsymbol{\xi}}$ denotes the functional derivative of $I[\boldsymbol{\xi}]$ and $\frac{\partial G}{\partial \mathbb{J}}$ and $\frac{\partial G}{\partial r} = \frac{\partial G}{\partial \det(\mathbb{J})}$ are the derivatives of G with respect to its first and second arguments.¹ Notice that this equation is defined for the inverse coordinate transformation $\boldsymbol{\xi}$. Since the computational mesh of Ω_c is typically given while the physical mesh of Ω is to be determined, it is common to transform the mesh equation (4) by exchanging the roles of the independent variable $\mathbf{x}$ and the dependent variable $\boldsymbol{\xi}$ and then discretize it on the computational mesh using finite difference or finite element methods. The resulting (nonlinear) algebraic system is solved for the physical mesh.

Although (4) is simple, the transformed mesh equation (after exchanging the roles of $\mathbf{x}$ and $\boldsymbol{\xi}$) becomes complicated and a serious effort is needed to implement its finite difference or finite element discretization on computer. The process can be simplified by some degree by first discretizing (4) directly on the physical mesh (which is normally not uniform and not rectangular), solving the resulting equations for the computational mesh, and then obtaining the new physical mesh by interpolation (see the next section for the detailed discussion for a similar procedure). However, the implementation of a finite difference or finite element discretization of (4) still requires a non-trivial effort. This is the motivation for us to seek a simpler discretization and a simpler implementation (see the next section).

Generally speaking, (4) is highly nonlinear. A useful strategy to solve such a nonlinear system is to use a time-varying approach or a moving mesh PDE (MMPDE) approach [13, 14] in the context of dynamical mesh adaptation. An MMPDE is defined as a gradient flow equation of the functional (1), i.e.,

$$\frac{\partial \boldsymbol{\xi}}{\partial t} = -\frac{P}{\tau} \frac{\delta I}{\delta \boldsymbol{\xi}}, \quad (5)$$

where t is the quasi-time, $\tau > 0$ is a constant parameter used to adjust the scale of mesh movement, and P is a positive balancing function. When it is desired to compute the physical

¹See Sect. 3.2 for the notation of scalar-by-matrix derivatives.

coordinates directly, we can combine (5) with the identity

$$\frac{\partial \mathbf{x}}{\partial t} + \mathbb{J}^{-1} \frac{\partial \boldsymbol{\xi}}{\partial t} = 0$$

to obtain

$$\frac{\partial \mathbf{x}}{\partial t} = \frac{P}{\tau} \mathbb{J}^{-1} \frac{\partial I}{\partial \boldsymbol{\xi}}.$$

3. Direct discretization and analytical formulas for derivatives of the discretized functional

3.1. Direct discretization

Denote the physical and computational meshes by $\mathcal{T}_h$ and $\mathcal{T}_{c,h}$ and assume they have the same numbers of elements and vertices and the same connectivity. Since we consider only simplicial meshes, for any element $K \in \mathcal{T}_h$ there exist a corresponding element $K_c \in \mathcal{T}_{c,h}$ and an invertible affine mapping $F_K: K_c \rightarrow K$ such that $K = F_K(K_c)$. With this notation, we approximate the functional (1) directly as

$$\begin{aligned} I[\boldsymbol{\xi}] &= \int_{\Omega} G(\mathbb{J}, \det(\mathbb{J}), \mathbb{M}, \mathbf{x}) d\mathbf{x} = \sum_{K \in \mathcal{T}_h} \int_K G(\mathbb{J}, \det(\mathbb{J}), \mathbb{M}, \mathbf{x}) d\mathbf{x} \\ &\approx \sum_{K \in \mathcal{T}_h} |K| G\left((F'_K)^{-1}, \det(F'_K)^{-1}, \mathbb{M}(\mathbf{x}_K), \mathbf{x}_K\right), \end{aligned} \quad (6)$$

where $|K|$ and $\mathbf{x}_K$ are the volume and the center of K and F'_K is the Jacobian matrix of F_K .

Note that in (6) we have used the mid-point quadrature formula for the involved integrals and approximated $\mathbb{J}$ using $(F'_K)^{-1}$ (instead of directly discretizing $\mathbb{J}$ on the mesh). The latter enables the discretized functional to preserve the basic geometric structure of the underlying functional. For example, the integrand of functional (2) is the trace of the matrix $\mathbb{J}\mathbb{M}^{-1}\mathbb{J}^T$. For this functional, the summand of (6) is the product of the volume of K and the trace of the matrix $(F'_K)^{-1}\mathbb{M}^{-1}(\mathbf{x}_K)(F'_K)^{-T}$, an approximation of $\mathbb{J}\mathbb{M}^{-1}\mathbb{J}^T$ on K . For functional (3), the summand of (6) is a linear combination of the terms corresponding to the alignment and equidistribution conditions as in the continuous functional. The preservation of the geometric properties is useful in preventing strong decoupling or loss of integral constraints satisfied by the underlying functional (cf. Castillo [4]). Moreover, as we will see in (17), F'_K can be computed using the edge matrices of K and K_c (cf. (18)). Therefore, no direct discretization of derivatives is involved in (6).

Notice that each summand in (6) is a function of the coordinates of vertices of K_c , i.e.,

$$I_K(\boldsymbol{\xi}_0^K, \dots, \boldsymbol{\xi}_d^K) = G\left((F'_K)^{-1}, \det(F'_K)^{-1}, \mathbb{M}(\mathbf{x}_K), \mathbf{x}_K\right), \quad (7)$$

while the sum is a function of the coordinates of all vertices of the mesh $\mathcal{T}_{c,h}$,

$$I_h(\boldsymbol{\xi}_1, \dots, \boldsymbol{\xi}_{N_v}) = \sum_{K \in \mathcal{T}_h} |K| I_K(\boldsymbol{\xi}_0^K, \dots, \boldsymbol{\xi}_d^K). \quad (8)$$

The key to our approach is to find the derivatives of I_h with respect to $\boldsymbol{\xi}_1, \dots, \boldsymbol{\xi}_{N_v}$. This is done by obtaining and assembling the elementwise derivatives of I_K with respect to $\boldsymbol{\xi}_0^K, \dots, \boldsymbol{\xi}_d^K$. We first recall some notation and results for scalar-by-matrix derivatives.

3.2. Scalar-by-matrix derivatives

Let $f = f(A)$ be a scalar function of a matrix $A \in \mathbb{R}^{m \times n}$. The scalar-by-matrix derivative of f with respect to A is defined as

$$\frac{\partial f}{\partial A} = \begin{bmatrix} \frac{\partial f}{\partial A_{11}} & \cdots & \frac{\partial f}{\partial A_{m1}} \\ \vdots & & \vdots \\ \frac{\partial f}{\partial A_{1n}} & \cdots & \frac{\partial f}{\partial A_{mn}} \end{bmatrix}_{n \times m} \quad \text{or} \quad \left(\frac{\partial f}{\partial A} \right)_{i,j} = \frac{\partial f}{\partial A_{j,i}}. \quad (9)$$

The chain rule of differentiation involving matrices (with respect to a real parameter t) is

$$\frac{\partial f}{\partial t} = \sum_{ij} \frac{\partial f}{\partial A_{j,i}} \frac{\partial A_{j,i}}{\partial t} = \sum_{ij} \left(\frac{\partial f}{\partial A} \right)_{i,j} \frac{\partial A_{j,i}}{\partial t} = \text{tr} \left(\frac{\partial f}{\partial A} \frac{\partial A}{\partial t} \right). \quad (10)$$

Hereafter, we always assume that (i) arguments of the matrix trace ($\text{tr}(\cdot)$) and determinant ($\det(\cdot)$) are square matrices; (ii) matrix products are meaningful; (iii) matrices are invertible when their inverses are involved.

The following four lemmas serve as the basic tools in the application of scalar-by-matrix differentiation. The first two can be verified directly while the third and fourth can be proven using the determinant expansion by minors and by differentiating the identity $AA^{-1} = I$, respectively.

Lemma 3.1.

$$\begin{aligned} \text{tr}(A^T) &= \text{tr}(A), \\ \text{tr}(AB) &= \text{tr}(BA), \\ \text{tr}(ABC) &= \text{tr}(CAB) = \text{tr}(BCA). \end{aligned}$$

Lemma 3.2.

$$\frac{\partial \text{tr}(A)}{\partial A} = I.$$

Lemma 3.3.

$$\frac{\partial \det(A)}{\partial A} = \det(A) A^{-1}.$$

Lemma 3.4.

$$\frac{\partial A^{-1}}{\partial t} = -A^{-1} \frac{\partial A}{\partial t} A^{-1}.$$

We now derive several identities which we need in our application.

Corollary 3.1. *If a symmetric matrix $\mathbb{M}$ is independent of A , then*

$$\frac{\partial \text{tr}(A\mathbb{M}A^T)}{\partial A} = 2\mathbb{M}A^T, \quad (11)$$

$$\frac{\partial \text{tr}(A^{-T}\mathbb{M}^{-1}A^{-1})}{\partial A} = -2A^{-1}A^{-T}\mathbb{M}^{-1}A^{-1}. \quad (12)$$

Moreover, if A is independent of $\mathbb{M}$, then

$$\frac{\partial \text{tr}(A\mathbb{M}A^T)}{\partial \mathbb{M}} = A^T A, \quad (13)$$

$$\frac{\partial \text{tr}(A\mathbb{M}^{-1}A^T)}{\partial \mathbb{M}} = -\mathbb{M}^{-1}A^T A \mathbb{M}^{-1}. \quad (14)$$

Proof. Let t be an entry of A . Using Lemmas 3.1 and 3.2, we have

$$\begin{aligned}\frac{\partial \operatorname{tr}(A\mathbb{M}A^T)}{\partial t} &= \operatorname{tr} \left(\frac{\partial \operatorname{tr}(A\mathbb{M}A^T)}{\partial (A\mathbb{M}A^T)} \frac{\partial (A\mathbb{M}A^T)}{\partial t} \right) = \operatorname{tr} \left(\frac{\partial (A\mathbb{M}A^T)}{\partial t} \right) \\ &= \operatorname{tr} \left(\frac{\partial A}{\partial t} \mathbb{M}A^T + A\mathbb{M} \frac{\partial A^T}{\partial t} \right) = \operatorname{tr} \left(\frac{\partial A}{\partial t} \mathbb{M}A^T \right) + \operatorname{tr} \left(A\mathbb{M} \frac{\partial A^T}{\partial t} \right) \\ &= \operatorname{tr} \left(\mathbb{M}A^T \frac{\partial A}{\partial t} \right) + \operatorname{tr} \left(\frac{\partial A}{\partial t} \mathbb{M}A^T \right) = \operatorname{tr} \left(2\mathbb{M}A^T \frac{\partial A}{\partial t} \right).\end{aligned}$$

From the chain rule (10), this gives (11).

Moreover, using Lemmas 3.1 and 3.4 and identity (11), we have

$$\begin{aligned}\frac{\partial \operatorname{tr}(A^{-T}\mathbb{M}^{-1}A^{-1})}{\partial t} &= \operatorname{tr} \left(\frac{\partial \operatorname{tr}(A^{-T}\mathbb{M}^{-1}A^{-1})}{\partial A^{-T}} \frac{\partial A^{-T}}{\partial t} \right) \\ &= \operatorname{tr} \left(2\mathbb{M}^{-1}A^{-1} \frac{\partial A^{-T}}{\partial t} \right) = \operatorname{tr} \left(2\mathbb{M}^{-1}A^{-1}(-A^{-1} \frac{\partial A^T}{\partial t} A^{-T}) \right) \\ &= \operatorname{tr} \left(-2A^{-1} \frac{\partial A}{\partial t} A^{-1} A^{-T} \mathbb{M}^{-1} \right) = \operatorname{tr} \left(-2A^{-1} A^{-T} \mathbb{M}^{-1} A^{-1} \frac{\partial A}{\partial t} \right),\end{aligned}$$

which gives (12).

Identities (13) and (14) can be proven similarly. $\square$

Using the above results, we can find the expressions for $\frac{\partial G}{\partial \mathbb{J}}$ and $\frac{\partial G}{\partial r}$, which are needed to compute the derivatives of the discrete functional (6), for functionals (2) and (3).

Example 3.1 (generalized Winslow's functional). For the functional (2) we have

$$\begin{cases} \frac{\partial G}{\partial \mathbb{J}} = 2\mathbb{M}^{-1}\mathbb{J}^T, \\ \frac{\partial G}{\partial r} = 0. \end{cases} \quad (15)$$

Example 3.2 (Huang's functional). For the functional (3) we have

$$\begin{cases} \frac{\partial G}{\partial \mathbb{J}} = dp\theta \sqrt{\det(\mathbb{M})} \left(\operatorname{tr}(\mathbb{J}\mathbb{M}^{-1}\mathbb{J}^T) \right)^{\frac{dp}{2}-1} \mathbb{M}^{-1}\mathbb{J}^T, \\ \frac{\partial G}{\partial r} = p(1-2\theta)d^{\frac{dp}{2}} \det(\mathbb{M})^{\frac{1-p}{2}} \det(\mathbb{J})^{p-1}. \end{cases} \quad (16)$$

3.3. Analytical formulas for derivatives of the discretized functional

The coordinates $\mathbf{x}_0^K, \dots, \mathbf{x}_d^K$ of the vertices of K and the coordinates $\boldsymbol{\xi}_0^K, \dots, \boldsymbol{\xi}_d^K$ of the vertices of K_c are related by

$$\mathbf{x}_i^K - \mathbf{x}_0^K = F'_K(\boldsymbol{\xi}_i^K - \boldsymbol{\xi}_0^K), \quad i = 1, \dots, d,$$

or, in matrix form,

$$[\mathbf{x}_1^K - \mathbf{x}_0^K, \dots, \mathbf{x}_d^K - \mathbf{x}_0^K] = F'_K[\boldsymbol{\xi}_1^K - \boldsymbol{\xi}_0^K, \dots, \boldsymbol{\xi}_d^K - \boldsymbol{\xi}_0^K].$$

Thus,

$$F'_K = E_K \hat{E}_K^{-1} \quad \text{and} \quad (F'_K)^{-1} = \hat{E}_K E_K^{-1}, \quad (17)$$

where E_K and $\hat{E}_K$ are the edge matrices for K and K_c ,

$$E_K = [\mathbf{x}_1^K - \mathbf{x}_0^K, \dots, \mathbf{x}_d^K - \mathbf{x}_0^K] \quad \text{and} \quad \hat{E}_K = [\boldsymbol{\xi}_1^K - \boldsymbol{\xi}_0^K, \dots, \boldsymbol{\xi}_d^K - \boldsymbol{\xi}_0^K]. \quad (18)$$

We now derive the derivatives of I_K . Using Lemmas 3.1 and 3.3 and (17), we have

$$\begin{aligned} \frac{\partial I_K}{\partial t} &= \text{tr} \left(\frac{\partial G}{\partial \mathbb{J}} \frac{\partial (F'_K)^{-1}}{\partial t} \right) + \frac{\partial G}{\partial r} \frac{\partial \det(F'_K)^{-1}}{\partial t} \\ &= \text{tr} \left(\frac{\partial G}{\partial \mathbb{J}} \frac{\partial \hat{E}_K}{\partial t} E_K^{-1} \right) + \frac{\partial G}{\partial r} \det(E_K)^{-1} \frac{\partial \det(\hat{E}_K)}{\partial t} \\ &= \text{tr} \left(E_K^{-1} \frac{\partial G}{\partial \mathbb{J}} \frac{\partial \hat{E}_K}{\partial t} \right) + \frac{\partial G}{\partial r} \frac{\det(\hat{E}_K)}{\det(E_K)} \text{tr} \left(\hat{E}_K^{-1} \frac{\partial \hat{E}_K}{\partial t} \right), \end{aligned} \quad (19)$$

where

$$\begin{aligned} \frac{\partial G}{\partial \mathbb{J}} &= \frac{\partial G}{\partial \mathbb{J}}(\hat{E}_K E_K^{-1}, \frac{\det(\hat{E}_K)}{\det(E_K)}, \mathbb{M}(\mathbf{x}_K), \mathbf{x}_K), \\ \frac{\partial G}{\partial r} &= \frac{\partial G}{\partial r}(\hat{E}_K E_K^{-1}, \frac{\det(\hat{E}_K)}{\det(E_K)}, \mathbb{M}(\mathbf{x}_K), \mathbf{x}_K). \end{aligned}$$

Letting t be any entry of the matrix $[\boldsymbol{\xi}_1^K, \dots, \boldsymbol{\xi}_d^K]$, we notice that

$$\frac{\partial \hat{E}_K}{\partial t} = \frac{\partial [\boldsymbol{\xi}_1^K, \dots, \boldsymbol{\xi}_d^K]}{\partial t}.$$

Combining this with (19), we get

$$\frac{\partial I_K}{\partial [\boldsymbol{\xi}_1^K, \dots, \boldsymbol{\xi}_d^K]} = E_K^{-1} \frac{\partial G}{\partial \mathbb{J}} + \frac{\partial G}{\partial r} \frac{\det(\hat{E}_K)}{\det(E_K)} \hat{E}_K^{-1}. \quad (20)$$

Moreover, for $j = 1, \dots, d$, from (19) and the equality

$$\frac{\partial \hat{E}_K}{\partial \xi_0^{K(j)}} = - \sum_{i=1}^d \frac{\partial \hat{E}_K}{\partial \xi_i^{K(j)}},$$

where $\xi_i^{K(j)}$ denotes the j^{th} component of $\boldsymbol{\xi}_i^K$, we have

$$\begin{aligned} \frac{\partial I_K}{\partial \xi_0^{K(j)}} &= - \sum_{i=1}^d \left[\text{tr} \left(\frac{\partial G}{\partial \mathbb{J}} \frac{\partial \hat{E}_K}{\partial \xi_i^{K(j)}} E_K^{-1} \right) + \frac{\partial G}{\partial r} \frac{\det(\hat{E}_K)}{\det(E_K)} \text{tr} \left(\hat{E}_K^{-1} \frac{\partial \hat{E}_K}{\partial \xi_i^{K(j)}} \right) \right] \\ &= - \sum_{i=1}^d \left(\frac{\partial I_K}{\partial [\boldsymbol{\xi}_1^K, \dots, \boldsymbol{\xi}_d^K]} \right)_{i,j}, \end{aligned}$$

which gives

$$\frac{\partial I_K}{\partial \xi_0^K} = -e^T \frac{\partial I_K}{\partial [\boldsymbol{\xi}_1^K, \dots, \boldsymbol{\xi}_d^K]}, \quad e = [1, \dots, 1]^T. \quad (21)$$

To summarize, the derivatives of I_h with respect to $\boldsymbol{\xi}_1, \dots, \boldsymbol{\xi}_{N_v}$ are computed as follows:

- (i) Compute the edge matrices E_K , $\hat{E}_K$ and their inverses from (18);
- (ii) Compute F'_K and its inverse via (17) and quantities $\frac{\partial G}{\partial \mathbb{J}}$ and $\frac{\partial G}{\partial r}$ through (15) or (16);
- (iii) Compute the derivatives of I_K with respect to $\xi_0^K, \dots, \xi_d^K$ through (20) and (21);
- (iv) Finally, the derivatives of I_h with respect to $\xi_1, \dots, \xi_{N_v}$ are obtained by assembling the element-wise derivatives (cf. (8)).

The mesh equation for $\xi_1, \dots, \xi_{N_v}$ reads as

$$\frac{\partial I_h}{\partial[\xi_1, \dots, \xi_{N_v}]} = 0. \quad (22)$$

This equation is analytical and, as for the standard finite element computation, the element-wise derivatives can be computed in parallel for all elements and then assembled together to form the global derivatives. Moreover, the Jacobian matrix for (22) is sparse. Its analytical expression is harder to obtain but its finite difference approximation can be computed in parallel as well.

Note that equation (22) can be highly nonlinear. As in the continuous situation, we can use the MMPDE approach (cf. (5)), i.e.,

$$\frac{\partial[\xi_1, \dots, \xi_{N_v}]}{\partial t} = -\frac{1}{\tau} \left(\frac{\partial I_h}{\partial[\xi_1, \dots, \xi_{N_v}]} \right)^T P, \quad (23)$$

where the balancing factor P is now a diagonal $N_v \times N_v$ matrix. Since mesh concentration should not be affected by scaling transformations of $\mathbb{M}$, we choose P such that both sides of (23) are homogeneous in the dimension of $\mathbb{M}$. For example, from (15), (16) and (20) we find that the dimension of P is

$$[P] = \begin{cases} [\mathbb{M}], & \text{for functional (2)} \\ [\mathbb{M}]^{\frac{d(p-1)}{2}}, & \text{for functional (3)}, \end{cases}$$

where $[P]$ and $[\mathbb{M}]$ denote the dimension of P and $\mathbb{M}$, respectively. A reasonable choice for $[\mathbb{M}]$ is $[\mathbb{M}] = \det(\mathbb{M})^{\frac{1}{d}}$. Hence, we can choose

$$P = \text{diag}(P_1, \dots, P_{N_v}) \quad \text{with} \quad P_i = \begin{cases} \det(\mathbb{M}(\mathbf{x}_i))^{\frac{1}{d}}, & \text{for functional (2)} \\ \det(\mathbb{M}(\mathbf{x}_i))^{\frac{p-1}{2}}, & \text{for functional (3)}. \end{cases} \quad (24)$$

With this choice of P , the MMPDE (23) is invariant under the scaling transformation $\mathbb{M} \rightarrow c\mathbb{M}$ for any positive constant c .

The mesh equation (23) can be written more explicitly using local mesh velocities. From (8), we have

$$\frac{\partial I_h}{\partial \xi_i} = \sum_{K \in \omega_i} |K| \frac{\partial I_K}{\partial \xi_i},$$

where ω_i is the element patch associated with vertex ξ_i . From this we can rewrite (23) into

$$\frac{\partial \xi_i}{\partial t} = \frac{P_i}{\tau} \sum_{K \in \omega_i} |K| v_{iK}^K, \quad i = 1, \dots, N_v \quad (25)$$

where i_K and $\mathbf{v}_{i_K}^K$ are the local index and velocity of vertex $\boldsymbol{\xi}_i$ on the element K , respectively. The local velocities are defined as

$$\begin{bmatrix} (\mathbf{v}_0^K)^T \\ \vdots \\ (\mathbf{v}_d^K)^T \end{bmatrix} = -\frac{\partial I_K}{\partial [\boldsymbol{\xi}_0^K, \dots, \boldsymbol{\xi}_d^K]}.$$

From (20) and (21), we have

$$\begin{bmatrix} (\mathbf{v}_1^K)^T \\ \vdots \\ (\mathbf{v}_d^K)^T \end{bmatrix} = -E_K^{-1} \frac{\partial G}{\partial \mathbb{J}} - \frac{\partial G}{\partial r} \frac{\det(\hat{E}_K)}{\det(E_K)} \hat{E}_K^{-1}, \quad \mathbf{v}_0^K = -\sum_{i=1}^d \mathbf{v}_d^K. \quad (26)$$

The MMPDE (25) should be modified properly for boundary vertices. For example, if $\boldsymbol{\xi}_i$ is a fixed boundary vertex, we replace the corresponding equation by

$$\frac{\partial \boldsymbol{\xi}_i}{\partial t} = 0. \quad (27)$$

When $\boldsymbol{\xi}_i$ is allowed to move on a boundary curve (in 2D) or surface (in 3D) represented by

$$\phi(\boldsymbol{\xi}) = 0,$$

then the mesh velocity $\frac{\partial \boldsymbol{\xi}_i}{\partial t}$ needs to be modified such that its normal component along the curve or surface is zero, i.e.,

$$\nabla \phi(\boldsymbol{\xi}_i) \cdot \frac{\partial \boldsymbol{\xi}_i}{\partial t} = 0.$$

The MMPDE (25) (with suitable modifications for boundary vertices) can be integrated from t_n to t_{n+1} for the new computational mesh $\mathcal{T}_{c,h}$. Once $\mathcal{T}_{c,h}$ has been computed, the new physical mesh $\widetilde{\mathcal{T}}_h$ is obtained via linear interpolation: if the correspondence between $\mathcal{T}_{c,h}$ and $\mathcal{T}_h$ (current physical mesh) is

$$\mathbf{x} = \Phi_h(\boldsymbol{\xi}): \Omega_c \rightarrow \Omega \quad \text{and} \quad \mathcal{T}_h = \Phi_h(\mathcal{T}_{c,h}),$$

then the new physical mesh is given by

$$\widetilde{\mathcal{T}}_h = \Phi_h(\mathcal{T}_{c0,h}),$$

where $\mathcal{T}_{c0,h}$ is a given reference mesh of Ω_c . Typically, $\mathcal{T}_{c0,h}$ should be chosen as uniform as possible but this is not necessary, although the non-uniformity of $\mathcal{T}_{c0,h}$ will affect the resulting physical mesh (see Example 4.1 and Fig. 2). Moreover, $\mathcal{T}_{c0,h}$ does not have to have the same number of vertices, elements, or the same connectivity as $\mathcal{T}_h$. Hence, a two-level strategy can be used to improve the efficiency: a coarser mesh for the mesh equation and a finer mesh —obtained via linear interpolation— for the physical equation (e.g, see Huang [8]).

3.4. Preservation of coercivity and convexity

In Sect. 2 we have mentioned that both Winslow's and Huang's functionals are coercive and convex/polyconvex which guarantee the existence of minimizers. In the following, we show that these properties are preserved by the discretization discussed in the preceding subsections. To this end, we state a lemma which generalizes Corollary 3.1 and whose proof is straightforward.

Lemma 3.5. *If A , B , C are square matrices of the same size and A and C are independent of B , then*

$$\frac{\partial \operatorname{tr}(ABC)}{\partial B} = CA. \quad (28)$$

We first consider Winslow's functional. In this case, from (2) and (17) the discretized functional can be expressed as

$$I_h = \sum_K |K| I_K(\hat{E}_K), \quad I_K(\hat{E}_K) = \operatorname{tr}(\hat{E}_K S \hat{E}_K^T), \quad (29)$$

where $S = E_K^{-1} \mathbb{M}^{-1}(\mathbf{x}_K) E_K^{-T}$. Recall that $\mathbb{M}$ is assumed to be symmetric and uniformly positive definite. If we assume that the (current) physical mesh $\mathcal{T}_h$ has no degenerate elements, then E_K is nonsingular and S is uniformly (over all elements) positive definite. Hence, there exists a positive constant α (independent of K) such that

$$I_K(\hat{E}_K) \geq \alpha \operatorname{tr}(\hat{E}_K \hat{E}_K^T) \quad \forall K \in \mathcal{T}_h. \quad (30)$$

The above inequality is a discrete analogue of the coercivity condition for continuous functionals (cf. [11, (6.52)]). Moreover, for any $K \in \mathcal{T}_h$ and any edge matrix $E_\eta = [\boldsymbol{\eta}_1^K - \boldsymbol{\eta}_0^K, \dots, \boldsymbol{\eta}_d^K - \boldsymbol{\eta}_0^K]$,

$$\operatorname{tr} \left(\frac{\partial I_K}{\partial \hat{E}_K} E_\eta \right) = \operatorname{tr} (2S \hat{E}_K^T E_\eta) = 2 \operatorname{tr} (E_\eta^T \hat{E}_K S).$$

From this,

$$\begin{aligned} \operatorname{tr} \left(\frac{\partial \operatorname{tr} \left(\frac{\partial I_K}{\partial \hat{E}_K} E_\eta \right)}{\partial \hat{E}_K} E_\eta \right) &= 2 \operatorname{tr} \left(\frac{\partial \operatorname{tr} (E_\eta^T \hat{E}_K S)}{\partial \hat{E}_K} E_\eta \right) \\ &= 2 \operatorname{tr} (S E_\eta^T E_\eta) = 2 \operatorname{tr} (E_\eta S E_\eta^T) \geq 0, \end{aligned}$$

which is a discrete analogue of the convexity condition for continuous functionals (cf. [11, (6.53)]). Thus, I_h preserves the coercivity and convexity of Winslow's functional. It is noted that the latter property implies that I_h is a convex function of $\boldsymbol{\xi}_1, \dots, \boldsymbol{\xi}_{N_v}$. Hence, I_h , with or without suitable boundary conditions, has a unique minimizer.

For Huang's functional (3), I_K takes the form

$$\begin{aligned} I_K &= \theta \det(\mathbb{M}(\mathbf{x}_K))^{\frac{1}{2}} (\operatorname{tr}(\hat{E}_K S \hat{E}_K^T))^{\frac{dp}{2}} \\ &\quad + (1 - 2\theta) d^{\frac{dp}{2}} \det(\mathbb{M}(\mathbf{x}_K))^{\frac{1-p}{2}} \det(E_K)^{-p} \det(\hat{E}_K)^p. \end{aligned} \quad (31)$$

As for Winslow's functional, we can show that I_K preserves the coercivity and polyconvexity of the continuous functional for $0 < \theta \leq \frac{1}{2}$, $dp \geq 2$, and $p \geq 1$. Particularly, I_K is polyconvex in the sense that it is convex when considered as a function of $\hat{E}_K$ and $\det(\hat{E}_K)$. Note that I_K is not convex in general when considered as a function of $\hat{E}_K$. In the continuous situation, coercivity and polyconvexity imply the existence of minimizers of the functional (e.g., see [6]). However, it is unclear to the authors if this is true in the discrete situation.

4. Numerical examples

In this section we present examples to demonstrate the direct discretization and solution strategy discussed in the previous section. Unless otherwise stated, we use Huang's functional (3) with $\theta = \frac{1}{3}$ and $p = 2$ as the meshing functional. Moreover, the mesh equation with $\tau = 0.1$ is integrated from $t = 0$ to $t = 1$ using Matlab ODE solver *ode15s*.

Example 4.1 (2D, mesh smoothing). In the first example we demonstrate how our method can be used for mesh smoothing. To this end, we choose a mesh on $(0, 1) \times (0, 1)$ (Fig. 2a or Fig. 2d) as the reference computational mesh $\mathcal{T}_{c0,h}$ and the initial computational mesh $\mathcal{T}_{c,h}$. The initial physical mesh $\mathcal{T}_h$ is obtained by randomly perturbing the coordinates of the interior vertices of $\mathcal{T}_{c,h}$ (see Figs. 2b and 2e). The mesh is smoothed by integrating the mesh equation with $\mathbb{M} = I$. The final meshes obtained at $t = 1$ are shown in Figs. 2c and 2f. One can see that they are very smooth and almost identical to $\mathcal{T}_{c0,h}$. The latter is due to the fact that the optimal coordinate transformation for this example ($\mathbb{M} = I$ and $\Omega = \Omega_c$) is $\mathbf{x}(\boldsymbol{\xi}) = \boldsymbol{\xi}$ and therefore the final mesh is identical to the reference computational mesh $\mathcal{T}_{c0,h}$.

Example 4.2 (2D, sine wave). In this example we generate an adaptive mesh to minimize the L^2 interpolation error bound for

$$u(\mathbf{x}) = \tanh \left(-30 \left[y - \frac{1}{2} - \frac{1}{4} \sin(2\pi x) \right] \right)$$

in the domain $\Omega = (0, 1) \times (0, 1)$ (see [7, Sect. 3.2] on the choice of $\mathbb{M}$). Figure 3 shows the adaptive mesh and close-ins at the tip of the sine wave and in the middle of the domain.

Example 4.3 (2D, horseshoe domain). In this example we have different computational and physical domains: $\Omega_c = (0, 1) \times (0, 1)$ and Ω is given through the parametrization

$$\mathbf{x} = \begin{pmatrix} -(1 + \eta) \cos(\pi\xi) \\ [1 + (2R - 1)\eta] \sin(\pi\xi) \end{pmatrix} \quad \text{with } R = 4.5.$$

As metric tensors we consider $\mathbb{M} = I$ and

$$\mathbb{M}(\mathbf{x}) = 1 + \left(x_1^2 + \sqrt{(x_2 - 2R)^2 + 1 \times 10^{-8}} \right)^{-1}. \quad (32)$$

Figure 4 shows initial and adaptive physical meshes with $\mathbb{M} = I$ and $\mathbb{M}$ from (32) for the uniform 21×21 criss-cross computational mesh of Ω_c .

For this example, it is known that an improper discretization of Winslow's functional can produce folded meshes [18]. In Figs. 5 and 6 we show mesh examples for Winslow's and Huang's functionals with $\mathbb{M} = I$ and different uniform computational meshes (3×3 , 5×5

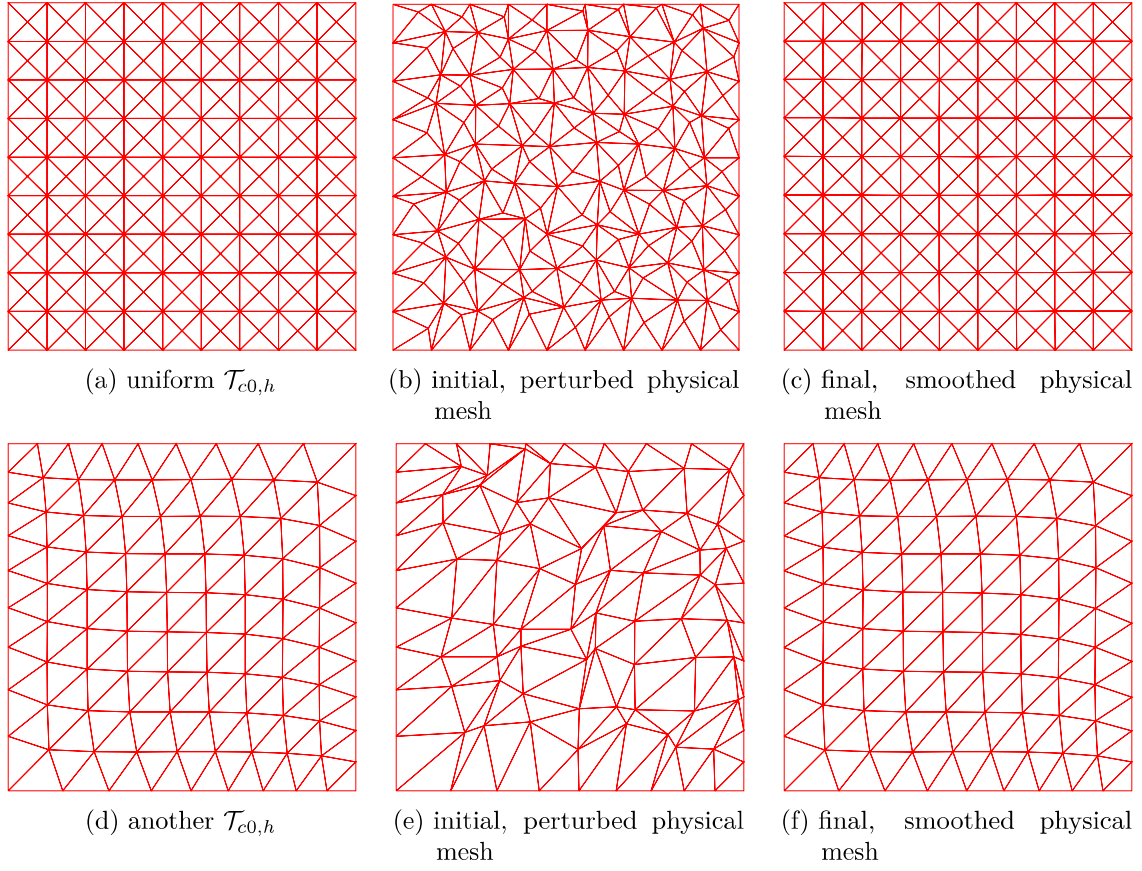


Figure 2: Mesh examples for mesh smoothing in Example 4.1

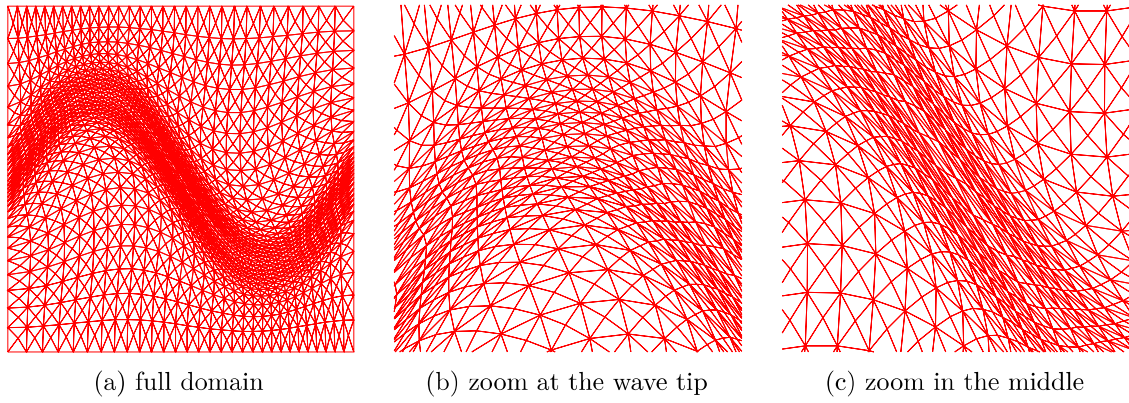


Figure 3: A 31×31 adaptive mesh for Example 4.2

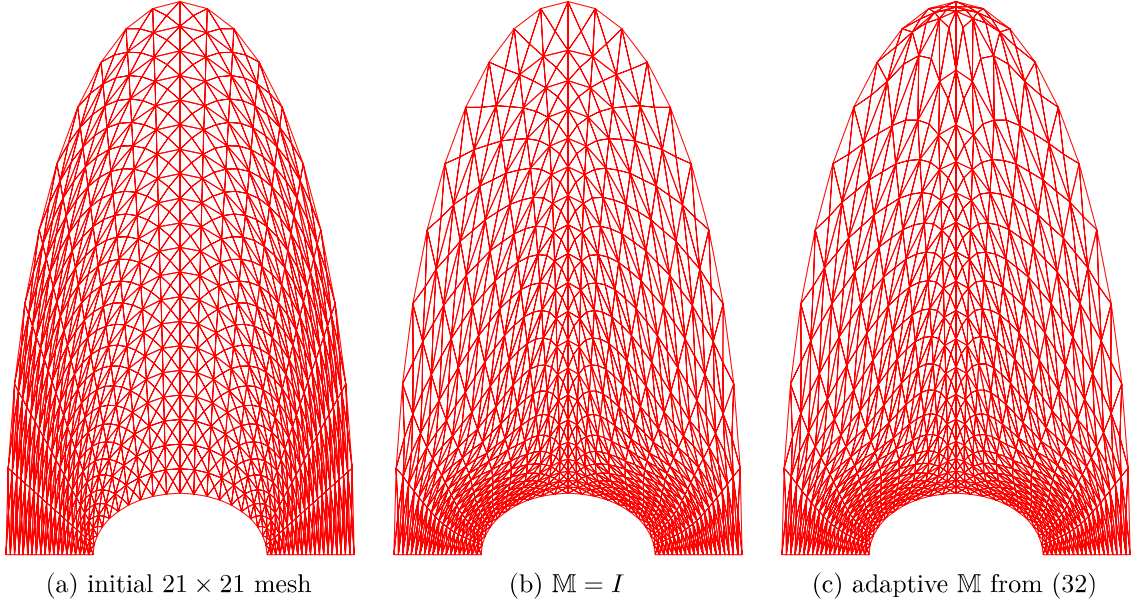


Figure 4: Mesh examples for Example 4.3

and 9×9). Notice that there is no mesh folding even on the coarsest meshes whereas other discretization methods can lead to mesh folding even on much finer meshes (cf. [18, Table 1]).

If the highly adaptive $\mathbb{M}$ from (32) is used, the situation becomes different and mesh folding does occur on a coarse mesh level. For Huang's functional the critical mesh size is about 17×17 (Fig. 7), whereas the critical mesh size for Winslow's functional is about 125×125 (Fig. 8).

Example 4.4 (3D, nine spheres). In this example we choose $\Omega = (-2, 2) \times (-2, 2) \times (-2, 2)$ and $\mathbb{M}$ to minimize the L^2 interpolation error bound (see [7] on the choice of $\mathbb{M}$) for

$$\begin{aligned}
u(\mathbf{x}) = & \tanh \left(30 \left[(x - 0.0)^2 + (y - 0.0)^2 + (z - 0.0)^2 - 0.1875 \right] \right) \\
& + \tanh \left(30 \left[(x - 0.5)^2 + (y - 0.5)^2 + (z - 0.5)^2 - 0.1875 \right] \right) \\
& + \tanh \left(30 \left[(x - 0.5)^2 + (y + 0.5)^2 + (z - 0.5)^2 - 0.1875 \right] \right) \\
& + \tanh \left(30 \left[(x + 0.5)^2 + (y - 0.5)^2 + (z - 0.5)^2 - 0.1875 \right] \right) \\
& + \tanh \left(30 \left[(x + 0.5)^2 + (y + 0.5)^2 + (z - 0.5)^2 - 0.1875 \right] \right) \\
& + \tanh \left(30 \left[(x - 0.5)^2 + (y - 0.5)^2 + (z + 0.5)^2 - 0.1875 \right] \right) \\
& + \tanh \left(30 \left[(x - 0.5)^2 + (y + 0.5)^2 + (z + 0.5)^2 - 0.1875 \right] \right) \\
& + \tanh \left(30 \left[(x + 0.5)^2 + (y - 0.5)^2 + (z + 0.5)^2 - 0.1875 \right] \right) \\
& + \tanh \left(30 \left[(x + 0.5)^2 + (y + 0.5)^2 + (z + 0.5)^2 - 0.1875 \right] \right).
\end{aligned}$$

Figure 9 shows an example of an adaptive mesh and cuts through the mesh in y - z plane.

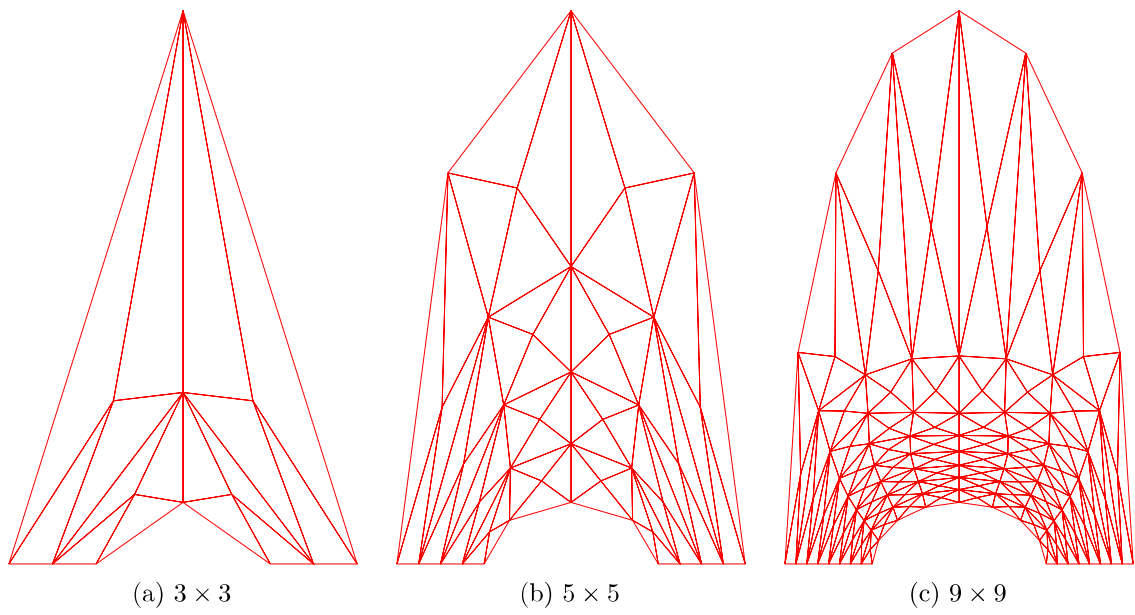


Figure 5: Example 4.3, $\mathbb{M} = I$, Huang's functional (3)

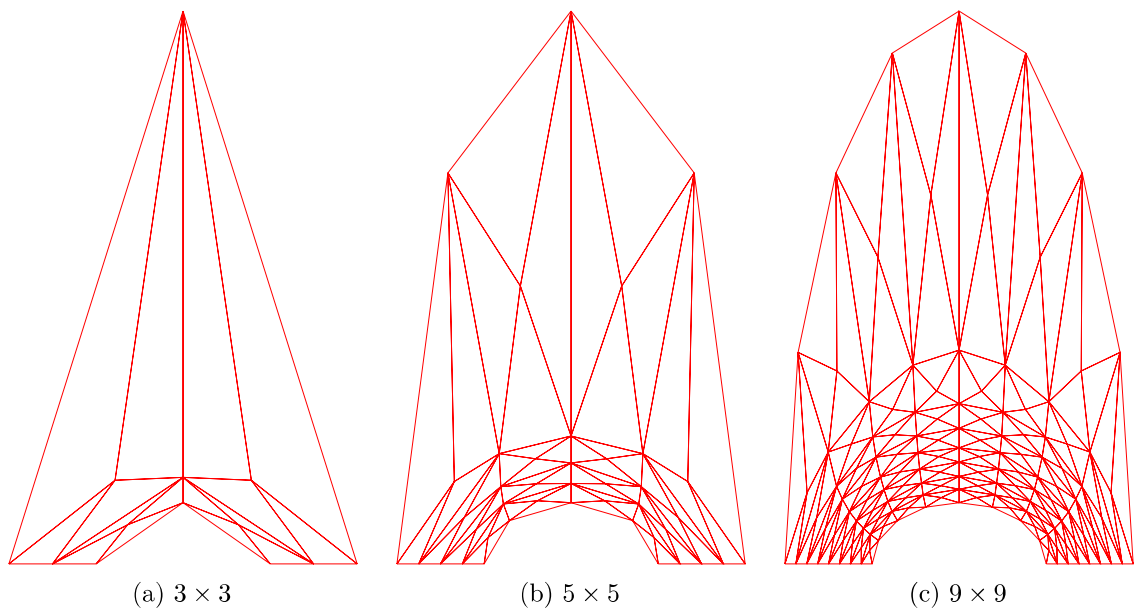


Figure 6: Example 4.3, $\mathbb{M} = I$, Winslow's functional (2)

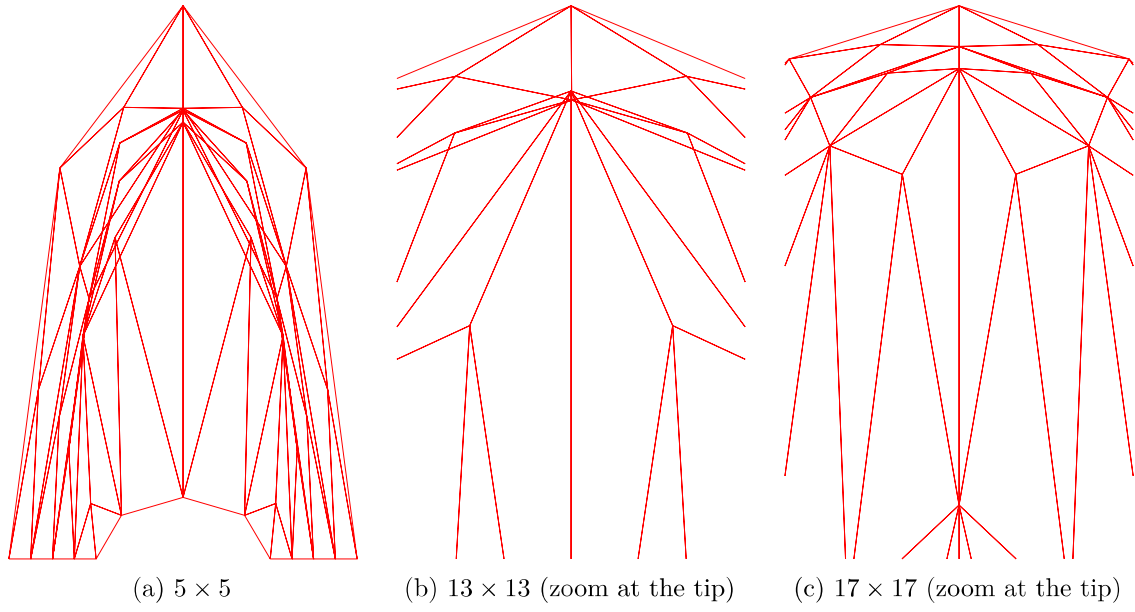


Figure 7: Example 4.3, adaptive M from (32), Huang's functional (3)

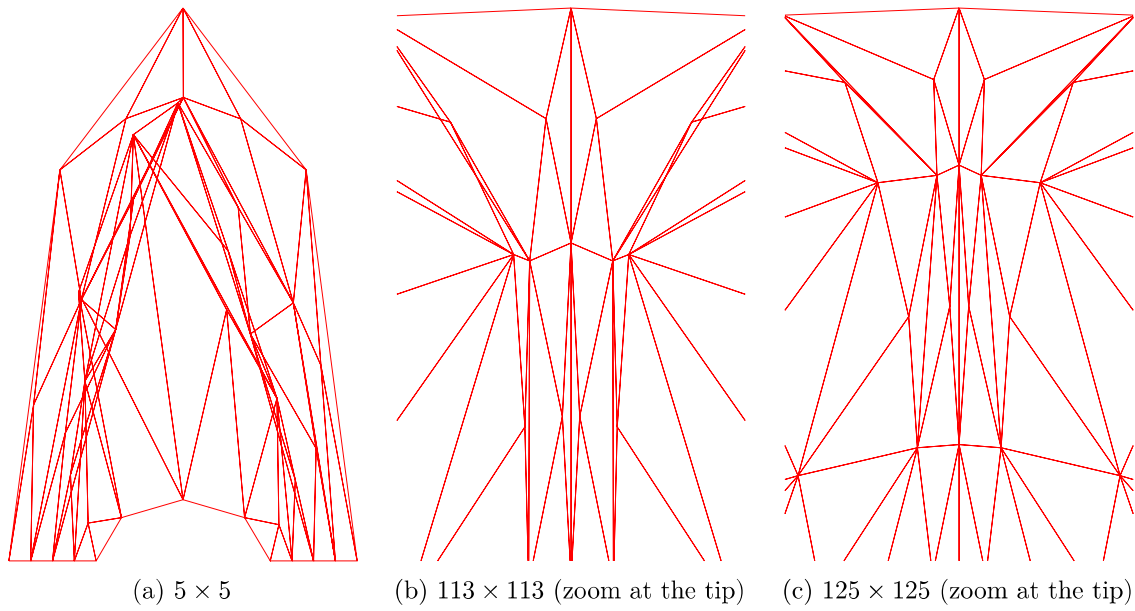
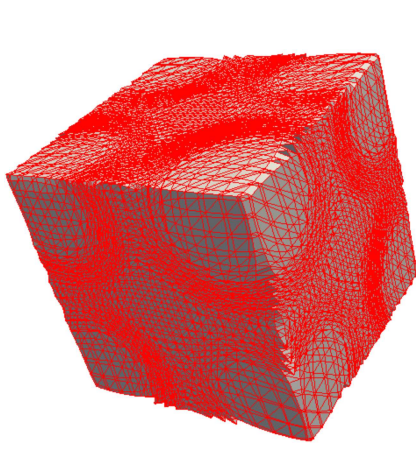
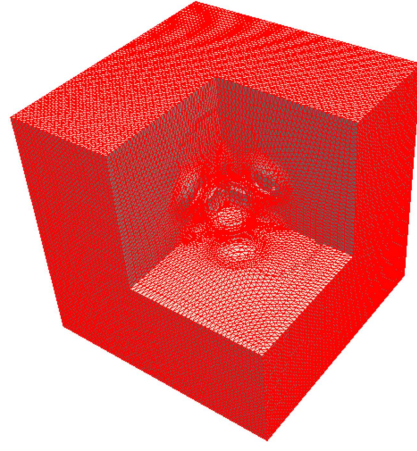


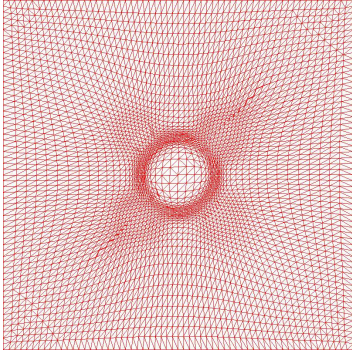
Figure 8: Example 4.3, adaptive M from (32), Winslow's functional (2)



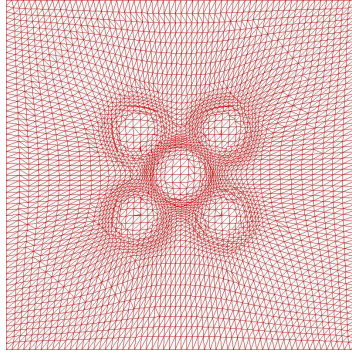
(a) inside-out cutaway



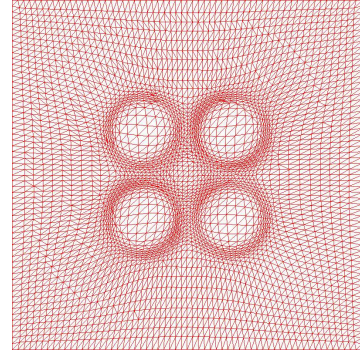
(b) cutaway



(c) y - z cut at $x = 0$

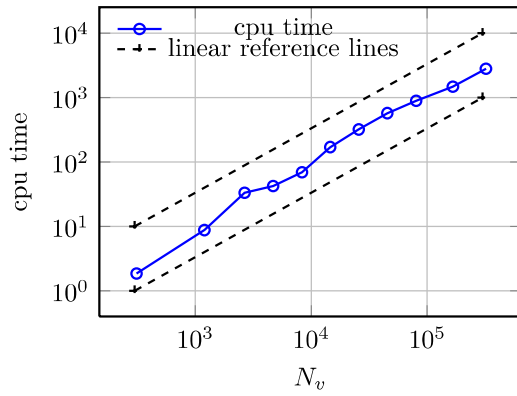


(d) y - z cut at $x = 0.25$

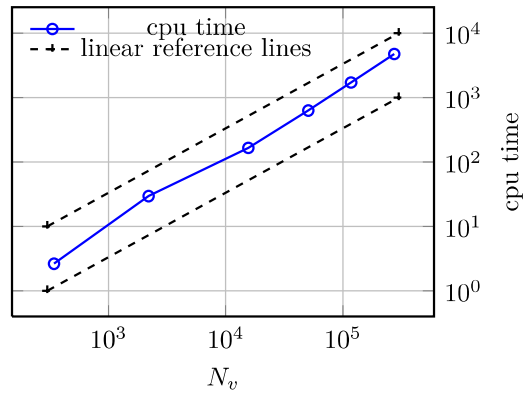


(e) y - z cut at $x = 0.5$

Figure 9: Adaptive mesh example and y - z plane cuts for Example 4.4



(a) Example 4.2 (2D)



(b) Example 4.4 (3D)

Figure 10: CPU time for mesh generation vs. number of vertices N_v

To test the computational cost of the new method, we measure the CPU time in dependence on the number of mesh nodes for Example 4.2 (2D) and Example 4.4 (3D) using a simple Matlab implementation running on a system with two Intel Xeons (E5-2665, 2.40 GHz). Numerical results suggest that the computational cost is linear in the number of mesh nodes for both 2D (Fig. 10a) and 3D (Fig. 10b) examples; it takes about 10 seconds of CPU time per 1 000 nodes.

5. Conclusions and further comments

In the previous sections we have proposed a direct discretization for a class of functionals used in variational mesh generation and adaptation. The meshing functionals are discretized directly on simplicial meshes and the Jacobian matrix of the continuous coordinate transformation is approximated by the Jacobian matrices of affine mappings between computational and physical elements (cf. (6)). Since the latter are computed from the edge matrices of the elements, there is no need for a direct discretization of derivatives of the coordinate transformation, a daunting task which is typically involved with nonuniform meshes.

Moreover, the proposed discretization has the advantage of preserving the basic geometric structure of the underlying continuous functional. In particular, it preserves the coercivity and convexity of Winslow's functional and the polyconvexity of Huang's functional (see Sect. 3.4).

A further advantage is the simple analytical formula for derivatives with respect to the coordinates of mesh vertices (cf. (20) and (21)), which allows a simple (and parallel) implementation. The computational cost is linear in the number of mesh nodes. A time-varying solution strategy of the mesh equation has also been discussed and applied to a number of numerical examples in Sect. 4. Since variational mesh generation and adaptation is employed as the base for a number of adaptive moving mesh, mesh smoothing, and refinement strategies, the result of this work can be used to develop simple implementations of those methods.

A. Derivatives of the discretized functional with respect to the physical coordinates

In Sect. 3 we used the computational coordinates ξ as unknown variables and obtained the physical mesh via linear interpolation. We can also use the physical coordinates $\mathbf{x}$ as unknown variables and obtain the physical mesh directly (this approach has been considered extensively in the existing literature, e.g., [11]). In the following we derive the formulas for the derivatives of the discretized functional with respect to the physical coordinates $\mathbf{x}$ for the convenience of users who prefer this approach. The formulas are slightly more complicated than those with respect to ξ and the metric tensor $\mathbb{M}$ has to be updated every time the physical mesh is updated.

To start with, notice that $|K| I_K$ is now a function of $\mathbf{x}_0^K, \dots, \mathbf{x}_d^K$. We have

$$\begin{aligned} \frac{\partial(|K| I_K)}{\partial t} &= G \frac{\partial |K|}{\partial t} + |K| \operatorname{tr} \left(\frac{\partial G}{\partial \mathbb{J}} \frac{\partial (F'_K)^{-1}}{\partial t} \right) + |K| \frac{\partial G}{\partial r} \frac{\partial \det(F'_K)^{-1}}{\partial t} \\ &\quad + |K| \sum_{k=1}^d \operatorname{tr} \left(\frac{\partial G}{\partial \mathbb{M}} \frac{\partial \mathbb{M}}{\partial x^{(k)}} \right) \frac{\partial x_K^{(k)}}{\partial t} + |K| \frac{\partial G}{\partial \mathbf{x}} \frac{\partial \mathbf{x}_K}{\partial t}. \end{aligned} \quad (33)$$

In the first term, $|K| = \frac{1}{d!} |\det(E_K)| = \frac{1}{d!} \det(E_K) \operatorname{sgn}(\det(E_K))$, where sgn is the sign function. From Lemma 3.3 we get

$$\frac{\partial |K|}{\partial t} = \frac{\operatorname{sgn}(\det(E_K))}{d!} \frac{\partial \det(E_K)}{\partial t} = \frac{|\det(E_K)|}{d!} \operatorname{tr} \left(E_K^{-1} \frac{\partial E_K}{\partial t} \right) = |K| \operatorname{tr} \left(E_K^{-1} \frac{\partial E_K}{\partial t} \right).$$

For the second term, from Lemmas 3.1 and 3.4 and equation (17) we obtain

$$\begin{aligned} \operatorname{tr} \left(\frac{\partial G}{\partial \mathbb{J}} \frac{\partial (F'_K)^{-1}}{\partial t} \right) &= \operatorname{tr} \left(\frac{\partial G}{\partial \mathbb{J}} \hat{E}_K \frac{\partial E_K^{-1}}{\partial t} \right) \\ &= -\operatorname{tr} \left(\frac{\partial G}{\partial \mathbb{J}} \hat{E}_K E_K^{-1} \frac{\partial E_K}{\partial t} E_K^{-1} \right) \\ &= -\operatorname{tr} \left(E_K^{-1} \frac{\partial G}{\partial \mathbb{J}} \hat{E}_K E_K^{-1} \frac{\partial E_K}{\partial t} \right). \end{aligned}$$

For the third term, from Lemma 3.2 we obtain

$$\frac{\partial G}{\partial r} \frac{\partial \det(F'_K)^{-1}}{\partial t} = -\frac{\partial G}{\partial r} \frac{\det(\hat{E}_K)}{\det(E_K)^2} \frac{\partial \det(E_K)}{\partial t} = -\frac{\partial G}{\partial r} \frac{\det(\hat{E}_K)}{\det(E_K)} \operatorname{tr} \left(E_K^{-1} \frac{\partial E_K}{\partial t} \right).$$

Combining the above results we get

$$\begin{aligned} \frac{\partial(|K| I_K)}{\partial[\mathbf{x}_1^K, \dots, \mathbf{x}_d^K]} &= G |K| E_K^{-1} - |K| E_K^{-1} \frac{\partial G}{\partial \mathbb{J}} \hat{E}_K E_K^{-1} - |K| \frac{\partial G}{\partial r} \frac{\det(\hat{E}_K)}{\det(E_K)} E_K^{-1} \\ &\quad + |K| (B_2)_{d \times d} + |K| (C_2)_{d \times d}, \end{aligned} \quad (34)$$

and

$$\begin{aligned} \frac{\partial(|K| I_K)}{\partial \mathbf{x}_0^K} &= -e^T \left[G |K| E_K^{-1} - |K| E_K^{-1} \frac{\partial G}{\partial \mathbb{J}} \hat{E}_K E_K^{-1} - |K| \frac{\partial G}{\partial r} \frac{\det(\hat{E}_K)}{\det(E_K)} E_K^{-1} \right] \\ &\quad + |K| (B_1)_{1 \times d} + |K| (C_1)_{1 \times d}, \end{aligned} \quad (35)$$

where B_1 and B_2 are associated with the fourth term in (33) and C_1 and C_2 are associated with the fifth term in (33).

We first derive C_1 and C_2 . Since $\mathbf{x}_K = \frac{1}{d+1} \sum_{k=0}^d \mathbf{x}_k^K$, we have

$$\frac{\partial G}{\partial \mathbf{x}} \frac{\partial \mathbf{x}_K}{\partial t} = \frac{1}{d+1} \sum_{k=0}^d \frac{\partial G}{\partial \mathbf{x}} \frac{\partial \mathbf{x}_k^K}{\partial t} = \frac{1}{d+1} \sum_{k=0}^d \sum_{l=1}^d \frac{\partial G}{\partial x^{(l)}} \frac{\partial x_k^{K(l)}}{\partial t}$$

and therefore

$$\frac{\partial G}{\partial \mathbf{x}} \frac{\partial \mathbf{x}_K}{\partial x_i^{K(j)}} = \frac{1}{d+1} \sum_{k=0}^d \sum_{l=1}^d \frac{\partial G}{\partial x^{(l)}} \frac{\partial x_k^{K(l)}}{\partial x_i^{K(j)}} = \frac{1}{d+1} \frac{\partial G}{\partial x^{(j)}}.$$

This gives

$$\begin{bmatrix} C_1 \\ C_2 \end{bmatrix} = \frac{1}{d+1} \begin{bmatrix} \frac{\partial G}{\partial \mathbf{x}} \\ \vdots \\ \frac{\partial G}{\partial \mathbf{x}} \end{bmatrix}. \quad (36)$$

The main difficulty in computing the fourth term and finding B_1 and B_2 is that $\mathbb{M}$ is typically defined on a background mesh as a piecewise linear function and therefore its derivatives do not exist on mesh facets, vertices, and edges. To avoid this difficulty, we assume that $\mathbb{M}$ has been interpolated from the background mesh to the current mesh $\mathcal{T}_h$ and the derivative $\frac{\partial \mathbb{M}}{\partial x^{(k)}}$ is approximated by that of the interpolating function, i.e.,

$$\mathbb{M} = \sum_{j=0}^d \mathbb{M}_{j,K} \phi_{j,K} \quad \text{and} \quad \frac{\partial \mathbb{M}}{\partial x^{(k)}} = \sum_{j=0}^d \mathbb{M}_{j,K} \frac{\partial \phi_{j,K}}{\partial x^{(k)}},$$

where $\mathbb{M}_{j,K}$ is the value of the metric tensor and $\phi_{j,K}$ is the linear basis function at the vertex $\mathbf{x}_j^K$. Then,

$$\begin{aligned} \sum_{k=1}^d \text{tr} \left(\frac{\partial G}{\partial \mathbb{M}} \frac{\partial \mathbb{M}}{\partial x^{(k)}} \right) \frac{\partial x_K^{(k)}}{\partial t} &= \sum_{k=1}^d \sum_{j=0}^d \text{tr} \left(\frac{\partial G}{\partial \mathbb{M}} \mathbb{M}_{j,K} \right) \frac{\partial \phi_{j,K}}{\partial x^{(k)}} \frac{\partial x_K^{(k)}}{\partial t} \\ &= \sum_{j=0}^d \text{tr} \left(\frac{\partial G}{\partial \mathbb{M}} \mathbb{M}_{j,K} \right) \frac{\partial \phi_{j,K}}{\partial \mathbf{x}} \frac{\partial \mathbf{x}_K}{\partial t}, \end{aligned}$$

which gives

$$\begin{bmatrix} B_1 \\ B_2 \end{bmatrix} = \frac{1}{d+1} \sum_{j=0}^d \text{tr} \left(\frac{\partial G}{\partial \mathbb{M}} \mathbb{M}_{j,K} \right) \begin{bmatrix} \frac{\partial \phi_{j,K}}{\partial \mathbf{x}} \\ \vdots \\ \frac{\partial \phi_{j,K}}{\partial \mathbf{x}} \end{bmatrix}. \quad (37)$$

The derivative $\frac{\partial \phi_{j,K}}{\partial \mathbf{x}} = (\nabla \phi_{j,K})^T$ is computed as follows. The basis functions satisfy

$$\sum_{j=0}^d \phi_{j,K} = 1 \quad \text{and} \quad \sum_{j=0}^d \mathbf{x}_j^K \phi_{j,K} = \mathbf{x}.$$

Eliminating $\mathbf{x}_{0,K}$ yields

$$\sum_{j=1}^d (\mathbf{x}_j^K - \mathbf{x}_0^K) \phi_{j,K} = \mathbf{x} - \mathbf{x}_0^K$$

and differentiating this with respect to $x^{(k)}$ gives

$$\sum_{j=1}^d (\mathbf{x}_j^K - \mathbf{x}_0^K) \frac{\partial \phi_{j,K}}{\partial x^{(k)}} = \mathbf{e}_k,$$

where $\mathbf{e}_k$ is the k^{th} unit vector of $\mathbb{R}^d$. Hence,

$$\begin{bmatrix} \frac{\partial \phi_{1,K}}{\partial \mathbf{x}} \\ \vdots \\ \frac{\partial \phi_{d,K}}{\partial \mathbf{x}} \end{bmatrix} = E_K^{-1} \quad \text{and} \quad \frac{\partial \phi_{0,K}}{\partial \mathbf{x}} = - \sum_{j=1}^d \frac{\partial \phi_{j,K}}{\partial \mathbf{x}}. \quad (38)$$

Like (25), we can write the MMPDE approach of the mesh equation in terms of mesh velocities. It reads as

$$\frac{\partial \mathbf{x}_i}{\partial t} = \frac{P_i}{\tau} \sum_{K \in \omega_i} |K| \mathbf{v}_{i,K}^K, \quad i = 1, \dots, N_v \quad (39)$$

where the balancing parameter P_i is defined in (24) and the local velocities are given by

$$\begin{aligned} \begin{bmatrix} (\mathbf{v}_1^K)^T \\ \vdots \\ (\mathbf{v}_d^K)^T \end{bmatrix} &= -G E_K^{-1} + E_K^{-1} \frac{\partial G}{\partial \mathbb{J}} \hat{E}_K E_K^{-1} + \frac{\partial G}{\partial r} \frac{\det(\hat{E}_K)}{\det(E_K)} E_K^{-1} \\ &\quad - \frac{1}{d+1} \sum_{j=0}^d \text{tr} \left(\frac{\partial G}{\partial \mathbb{M}} \mathbb{M}_{j,K} \right) \begin{bmatrix} \frac{\partial \phi_{j,K}}{\partial \mathbf{x}} \\ \vdots \\ \frac{\partial \phi_{j,K}}{\partial \mathbf{x}} \end{bmatrix} - \frac{1}{d+1} \begin{bmatrix} \frac{\partial G}{\partial \mathbf{x}} \\ \vdots \\ \frac{\partial G}{\partial \mathbf{x}} \end{bmatrix}, \end{aligned} \quad (40)$$

$$(\mathbf{v}_0^K)^T = - \sum_{k=1}^d (\mathbf{v}_k^K)^T - \sum_{j=0}^d \text{tr} \left(\frac{\partial G}{\partial \mathbb{M}} \mathbb{M}_{j,K} \right) \frac{\partial \phi_{j,K}}{\partial \mathbf{x}} - \frac{\partial G}{\partial \mathbf{x}}. \quad (41)$$

References

- [1] G. Beckett, J. A. MacKenzie, and M. L. Robertson. A moving mesh finite element method for the solution of two-dimensional Stefan problems. *J. Comput. Phys.* 168 (2001), pp. 500–518.
- [2] J. U. Brackbill and J. S. Saltzman. Adaptive zoning for singular problems in two dimensions. *J. Comput. Phys.* 46 (1982), pp. 342–368.
- [3] W. Cao, W. Huang, and R. D. Russell. An r -adaptive finite element method based upon moving mesh PDEs. *J. Comput. Phys.* 149 (1999), pp. 221–244.
- [4] J. Castillo. Mathematical aspects of variational grid generation I. *Numerical Grid Generation in Computational Fluid Dynamics*. Ed. by J. Hauser and C. Taylor. Pineridge Press Limited, Swansea, U.K., 1986.
- [5] A. S. Dvinsky. Adaptive grid generation from harmonic maps on Riemannian manifolds. *J. Comput. Phys.* 95 (1991), pp. 450–476.
- [6] L. C. Evans. *Partial Differential Equations*. Graduate Studies in Mathematics, Volume 19. Providence, Rhode Island: American Mathematical Society, 1998.
- [7] W. Huang. Metric tensors for anisotropic mesh generation. *J. Comput. Phys.* 204 (2005), pp. 633–665.
- [8] W. Huang. Practical aspects of formulation and solution of moving mesh partial differential equations. *J. Comput. Phys.* 171 (2001), pp. 753–775.

- [9] W. Huang. Variational mesh adaptation: isotropy and equidistribution. *J. Comput. Phys.* 174 (2001), pp. 903–924.
- [10] W. Huang and R. D. Russell. A high dimensional moving mesh strategy. *Appl. Numer. Math.* 26 (1998), pp. 63–76.
- [11] W. Huang and R. D. Russell. *Adaptive Moving Mesh Methods*. Applied Mathematical Sciences Series, Vol. 174. New York: Springer, 2011.
- [12] W. Huang and R. D. Russell. Moving mesh strategy based upon a gradient flow equation for two dimensional problems. *SIAM J. Sci. Comput.* 20 (1999), pp. 998–1015.
- [13] W. Huang, Y. Ren, and R. D. Russell. Moving mesh methods based on moving mesh partial differential equations. *J. Comput. Phys.* 113 (1994), pp. 279–290.
- [14] W. Huang, Y. Ren, and R. D. Russell. Moving mesh partial differential equations (MMPDEs) based upon the equidistribution principle. *SIAM J. Numer. Anal.* 31 (1994), pp. 709–730.
- [15] W. Huang, L. Kamenski, and R. D. Russell. A comparative numerical study of meshing functionals for variational mesh adaptation. *J. Math. Study* 48.2 (2015), pp. 168–186.
- [16] P. M. Knupp. Jacobian-weighted elliptic grid generation. *SIAM J. Sci. Comput.* 17 (1996), pp. 1475–1490.
- [17] P. M. Knupp and N. Robidoux. A framework for variational grid generation: conditioning the Jacobian matrix with matrix norms. *SIAM J. Sci. Comput.* 21 (2000), pp. 2029–2047.
- [18] P. Knupp and R. Luczak. Truncation error in grid generation: A case study. *Numer. Meth. PDEs* 11 (1995), 561–571.
- [19] P. Knupp and S. Steinberg. *Fundamentals of Grid Generation*. Boca Raton: CRC Press, 1994.
- [20] R. Li, T. Tang, and P. W. Zhang. Moving mesh methods in multiple dimensions based on harmonic maps. *J. Comput. Phys.* 170 (2001), pp. 562–588.
- [21] V. D. Liseikin. *Grid Generation Methods*. Berlin: Springer, 1999.
- [22] J. F. Thompson, Z. A. Warsi, and C. W. Mastin. *Numerical Grid Generation: Foundations and Applications*. New York: North-Holland, 1985.
- [23] A. M. Winslow. *Adaptive mesh zoning by the equipotential method*. Tech. rep. UCID-19062. Lawrence Livermore Laboratory, 1981 (unpublished).